

PL Final

AAM-Based Abstract Interpretation in Haskell

Gerald Hoff

Electrical Engineering and Computer Science

Washington State University

Pullman, WA

gerald.hoff@wsu.edu

I. AAM IMPLEMENTATION

Overview

Abstracting Abstract Machines derives a static analysis from a concrete machine with two main ideas: bound the address space so the store stays finite, and store sets of values instead of single values so that merged addresses do not lose information. The result is a nondeterministic transition relation whose reachable states over-approximate every possible concrete execution.

This implementation (github) runs a CESK* machine over the pure λ -calculus. The object language has variables, lambdas, and applications, all encoded as a Haskell ADT with constructors `Ref`, `Lam`, and `(:@)`.

Machine State

A state is a five-tuple $\langle e, \rho, \hat{\sigma}, \kappa, t \rangle$: the control expression e , environment $\rho : \text{Var} \rightarrow \text{Addr}$, abstract store $\hat{\sigma} : \text{Addr} \rightarrow \mathcal{P}(\text{Storable})$, current continuation κ , and time stamp t . For CESK* continuations are *store-allocated*: each frame is stored at a `KAddr` address rather than kept on an explicit stack. This ensures that the stack never grows outside the store, so the analysis is finite.

Time and addresses. Time is a k -length call-string built from expressions. `tick` advances it by prepending the current expression and truncating:

```
tick :: Sigma -> Time
tick (e, _, _, _, t) = take k (e : t)
```

Binding addresses are `BAddr(x, t)` and continuation addresses are `KAddr(e, t)`. At $k = 0$ every call maps to the same address (0-CFA monovariance); at $k = 1$ addresses distinguish the most recent call site (1-CFA sensitivity).

Abstract store. Because addresses are finite, two different bindings may land at the same address. The store maps addresses to *sets* and uses a join-insert rather than overwrite:

```
(\!)! :: (Ord k, Lattice v)
=> (k -> v) -> [(k, v)] -> (k -> v)
f \! [(key, v)] = Map.insertWith (join) key v f
```

Lookup returns the empty set for missing keys, so the machine never crashes on unknown addresses.

Transition Rules

`step :: Sigma -> [Sigma]` returns a list of successors; multiple results arise when an address holds more than one value.

Variable lookup iterates over every closure at the variable's address, producing one successor per closure:

```
step state@(Ref x, rho, sigma, kappa, t) =
  [ (Lam lam, rho', sigma, kappa, t')
  | Clo (lam, rho') <-
    Set.toList (sigma !! (rho ! x)) ]
  where t' = tick state
```

Application allocates a fresh `KAddr`, saves the current continuation there, and pushes an `Ar` frame so the machine next evaluates the operator:

```
step state@(f :@ e, rho, sigma, kappa, t) =
  [ (f, rho, sigma', kappa', t') ]
  where
    t' = tick state
    a' = allocKont (f :@ e) t'
    sigma' = sigma oplus! [a' ==> s (Cont kappa)]
    kappa' = Ar (e, rho, a')
```

Call fires when both sides are evaluated (the `Fn` frame). A fresh `BAddr` is allocated for the parameter, the argument closure is joined into the store, and the saved continuation is retrieved.

```
step state@(Lam lam, rho, sigma,
  Fn (x ==> e, rho', a), t) =
  [ (e, rho' // [x ==> a'],
    sigma oplus! [a' ==> s (Clo (lam, rho))],
    kappa, t')
  | Cont kappa <- Set.toList (sigma !! a) ]
  where t' = tick state
        a' = allocBind x t'
```

Exploration and Tests

A worklist search collects all reachable states, skipping any already seen. Termination is guaranteed because the state space is finite. Final states match $(\text{Lam } _, _, _, \text{Mt}, _)$.

Four programs check correctness: identity applied to a lambda (`test1`), a higher-order function (`test2`), Church successor applied to Church zero (`test3`), and Ω . The latter should of course cycle through states without ever producing a final state, confirming the analysis terminates even on non-terminating programs.

II. COMPARISON WITH SUMMARY BASED PROGRAM ANALYSIS

The implementation of Abstracting Abstract Machines (AAM) described in this project provides a systematic way to derive static analyses from concrete operational semantics. This approach is in contrast to the summary-based interprocedural dataflow analysis framework which transforms dataflow problems into graph-reachability problems.

Methodological Foundations

AAM is fundamentally a derivational approach that begins with a concrete abstract machine, such as the CESK machine, and refactors it into a finite-state version by bounding the address space of the store. It relies on store-allocated continuations to eliminate recursive structures in the state-space, ensuring that the resulting abstract transition relation is finite and computable. In contrast, the summary-based approach described by Reps, Horwitz, and Sagiv focuses on solving a specific class of dataflow-analysis problems by converting them into a realizable-path reachability problem on an exploded supergraph. While AAM simulates the execution of a machine, the IFDS framework treats the program as a collection of procedure flow graphs and uses a tabulation algorithm to determine if a dataflow fact at one point can reach another point via a path that respects the call-return structure of the program.

Interprocedural Precision and the Stack

Both approaches aim for "precise" interprocedural analysis, but they handle the program stack differently:

- **AAM Strategy:** AAM achieves finiteness by allocating continuations (the stack) in the store. This allows the stack to be abstracted using the same mechanisms as variable bindings, where multiple continuation frames can be joined at a single abstract address. Precision is tuned through the use of time-stamps or call-strings, where a k -length string distinguishes different calling contexts (k-CFA).
- **Summary-Based Strategy:** The IFDS framework achieves precision by ensuring reachability only along "valid paths," where "returns" are matched with their corresponding "calls" using a context-free grammar of balanced parentheses. It utilizes "summary edges" at call sites to record the effects of a procedure call, allowing the analysis to avoid re-analyzing a procedure's body for every distinct call site.

Efficiency and Complexity

The IFDS framework offers strong theoretical guarantees on complexity. Specifically, it can solve distributive interprocedural problems in polynomial time, $O(ED^3)$, where E is the number of edges in the supergraph and D is the size of the dataflow fact set. This efficiency is made possible by the restriction that dataflow functions must distribute over the confluence operator (union or intersection). AAM is generally more computationally expensive because it explores a state-space that

tracks the history of the machine's execution. This produces more intensional view-tracking of variables at the cost of the complexity. However, AAM can be widened to a single-threaded global store to achieve polynomial-time performance similar to traditional k -CFA.

Generality and Expressiveness

A significant advantage of the AAM approach is its extreme generality. Because it is derived directly from the operational semantics, it scales naturally to handle complex language features such as first-class continuations, by-need evaluation, and garbage collection. For instance, AAM can soundly approximate the Ω -combinator or programs with side effects simply by applying the same structural abstraction to the relevant machine rules. The summary-based IFDS framework is more specialized. It is highly effective for "locally separable" or "distributive" problems (such as reaching definitions or live variables) but requires significant adaptation for problems that do not fit the distributive subset model. While the IDE (Interprocedural Distributive Environment) framework generalizes IFDS to handle some non-subset problems, it still relies on a distributive structure that may not easily capture the full range of intensional properties that an AAM-based machine can explore.

III. ACCELERATE TO A FIXPOINT

Accelerate is a Haskell EDSL for data-parallel GPU programming whose skeleton-based compiler translates array combinators into CUDA code. However, Accelerate disallows recursion because the code generator must see the complete computation structure at compile time. Users needing iterative algorithms must drive iteration from the host, relaunching kernels and crossing the CPU-GPU bus each step.

AAM's fixpoint machinery, instantiated with a domain of *array descriptors* rather than closures, can analyze iterative GPU loop bodies and compute the invariants a compiler needs. The same lattice-theoretic fixpoint that finitizes the lambda calculus can finitize an array dimension lattice.

Abstract Domain and Transfer Functions

The analysis operates over array descriptors: triples $(\hat{s}, p, f)$ of abstract shape, producer classification, and fusability. Abstract shapes form a two-element-height lattice: `ExactShape [n,m]` records known dimensions and `AnyShape` is top. Joining two identical exact shapes yields that shape; joining differing shapes yields `AnyShape`. Termination follows directly from this finite height.

Each array combinator has a transfer function over this domain. Shape-preserving operations (`map`, `zipWith`, `stencil`) return the input descriptor with updated metadata. Shape-changing operations reflect their semantics: `fold` removes the innermost dimension, `generate [n,m]` forces an exact output shape, and `filter` immediately widens to `AnyShape` because its output length is data-dependent. A pipeline of operations is analyzed by composing transfer functions left to right.

An `awhile` loop follows the AAM pattern: beginning from the initial descriptor, iterate $d' = d \sqcup f(d)$ until $d' = d$. Shape-preserving loops converge in zero widening steps; shape-changing loops stabilize in one step when the shape widens to `AnyShape`.

Compilation Strategy

When the analysis proves shape stability, the invariant is an `ExactShape` matching the initial allocation, the compiler emits a ping-pong buffer strategy: two device buffers allocated once, with a pointer swap replacing reallocation each iteration. The loop body becomes a persistent kernel, avoiding repeated launches and host synchronization. When stability cannot be proved, the compiler falls back to per-iteration reallocation. For a 1024×1024 Jacobi solver running 1000 iterations, this means 2 `cudaMalloc` calls versus 1001.

The analysis also tracks fusion information through the fixpoint. A body of fusable producers (`zipWith` then `map`) compiles as a single fused kernel; a body containing a consumer like `stencil` receives partial fusion.

Demonstration and Limitations

The implementation analyzes five programs: Jacobi relaxation, Mandelbrot iteration, a fixed generate-map loop, an iterative filter, and k-means. The first three are correctly identified as shape-stable and receive ping-pong code. The filter widens to `AnyShape` after one step; k-means, whose `fold` reduces dimensionality, is likewise classified as varying. In each case the analysis terminates and the generated CUDA pseudocode reflects the proven invariant.

This is a restricted prototype: the domain tracks only shape, not value ranges or index expressions, and loop bodies are flat pipelines rather than full Accelerate ASTs. Extending the domain to symbolic dimension expressions would widen the class of provably stable loops at the cost of slower convergence. This shows that AAM's fixpoint framework, applied to an array descriptor lattice, produces exactly the invariants a GPU compiler needs to choose between efficient and conservative memory strategies.

REFERENCES

- [1] D. Van Horn and M. Might, "Systematic Abstraction of Abstract Machines," *J. Functional Programming*, vol. 22, no. 4–5, pp. 705–746, 2012.
- [2] D. Van Horn and M. Might, "Abstracting Abstract Machines," *Proc. 15th ACM SIGPLAN Int. Conf. on Functional Programming (ICFP '10)*, pp. 101–112, 2010.
- [3] T. Reps, S. Horwitz, and M. Sagiv, "Precise Interprocedural Dataflow Analysis using Graph Reachability," *Proc. 22nd ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL '95)*, pp. 49–61, 1995.
- [4] T. L. McDonell, M. M. T. Chakravarty, G. Keller, and B. Lippmeier, "Optimising Purely Functional GPU Programs," *ACM SIGPLAN Notices*, vol. 48, no. 9, pp. 49–60, 2013.